

OpenWrt on the Nubia RedMagic 7 (NX679J)

A step-by-step reconstruction guide: firmware, bootloader unlock, blob extraction, build, flash to slot B, boot, return to Android, and safe revert

Reconstructed from the openwrt-nx679j repository (commit f9a92fc). This is a best-effort reconstruction, not the authors' original one-click pipeline: some build inputs are absent from the repository and are flagged as gaps throughout.

Reconstruction guide — gaps clearly flagged — brick risk is real

Target: OpenWrt 25.12.5 userland in a chroot on slot B, over the untouched vendor kernel 5.10.66.

Firmware: V311 (North America & Asia).

Report date: 4 October 2026

Read this first

WARNING

This procedure can **permanently brick the phone**. On this device the authors verified that Qualcomm EDL (emergency download, mode 9008) is **broken**: the Sahara handshake degenerates and no EDL command recovers it. That removes the normal last-resort recovery. **The only safety net is slot A** (stock Android + Magisk). If slot A is damaged while the bootloader is unlocked and EDL is dead, there may be no way back. Do not start unless you accept that risk, you own the device, and you have a full backup of everything on it.

What this guide is. A reconstruction assembled by reading the repository, which documents the project but does not ship a runnable end-to-end pipeline. Every step is labelled so you always know what you are standing on:

Label	Meaning
Verified in the repo	The authors ran this; commands/values come straight from their files
Reconstruction note	Reassembled from earlier artifacts or standard tooling; plausible but not their exact script
GAP	A required input or script is absent from the repository; you must supply or rebuild it

What you will and will not get. Following this faithfully reaches a booting OpenWrt with SSH on the USB gadget (the v10 milestone). The authors' final image (v180: modem without ModemManager, native UI, display standby) depends on build inputs that are not published, so full parity requires reconstruction work this guide can scope but not fully script.



Green = documented & runnable · Amber = reconstructable from earlier artifacts · Red = missing input

Figure 1 — The eight stages of this guide, colour-coded by how much the repository actually supports each one.

On the author’s rebuild-from-scratch claim. The repository’s own “Come ricostruire da zero” section (in DOCUMENTAZIONE.md) lays out a real, ordered rebuild path, and after re-reading it and the tree carefully this guide corrects an earlier under-count on our part: more is shipped than first stated. The **final modem design uses no ModemManager** — it is driven by a QRTR toolkit (qmi-qrtr-observed, chain.sh, rmnet-*, rmtfs, pd-mapper, tqftpserv) that is **prebuilt inside tools.tar**; the **native UI, kiosk3 and uinput-touch binaries are shipped** (with source); and the libqmi patches are present. What is *not* in the repo is the frozen live-rootfs base the final builder extracts. So the honest reading sits between the two positions: the port is **reconstructable to a functional equivalent** by following section 9 plus ESTRAZIONE-BLOB.md, but not **byte-identical**, because that one base artifact and a couple of convenience binaries must be rebuilt rather than copied.

Conventions. Host = your Linux PC. Device = the phone. adb/fastboot from Android platform-tools. The device’s USB gadget answers at 10.0.0.1 (SSH) once OpenWrt boots. Serial numbers shown as 3dbd**** are redacted; yours will differ.

Device and layout at a glance

Item	Value (as measured in the project)
Model / SoC	Nubia RedMagic 7, NX679J, Qualcomm SM8450 / Waipio (Snapdragon 8 Gen 1)
Vendor kernel	5.10.66-android12-9-00005-gf6e6376090be-ab8060604 (left untouched)
Stock firmware	NX679J_Z69_UN_ZML1S_V311 (North America & Asia), Android 12
Boot image size	boot_a / boot_b = 100,663,296 B (96 MiB fixed), Android header v4
Slots	A = stock Android + Magisk (recovery/oracle); B = the OpenWrt port
Bootloader state	unlocked / orange, secure:yes; A/B state lives in GPT attributes, not misc
Recovery path	slot A only — EDL (9008) is broken on this unit
OpenWrt	25.12.5 r33051-f5dae5ece4, target armsr/armv8, aarch64_generic (musl)

Slot A (intact) — recovery path



Slot B (modified) — the OpenWrt port



Shared firmware A=B (byte-identical): xbl, abl (LinuxLoader/UEFI), vbmeta, vendor_boot — NOT modified

Figure 2 — Partition and slot map. Only `boot_b` and `dtbo_b` are modified; the signed firmware chain is identical A=B and is never touched.

Partition by-name to block-device map (the project worked with these directly):

Partition	Block node	Use
boot_a / boot_b	sde13 / sde41 (259:25)	kernel + ramdisk; boot_b is the port
dtbo_a / dtbo_b	sde17 / sde45	device-tree overlays (44 entries); #35 patched for the panel
vendor_boot_a/_b	sde24 / sde52	vendor ramdisk + 9 concatenated DTBs
vbmeta*, xbl, abl, uefi	sdb1/sdc1, sde10/38, sde1/29	signed chain, identical A=B, not modified
super / userdata / persist	sda7 / sda12 / sda2	logical partitions, /data, calibrations
misc	sda3	BCB; a stray bootonce-bootloader here triggers the EDL loop
modem_a/_b, modemst1/2, fsg, fsc	sde6..., sdf2-sdf5	modem firmware and EFS calibrations
rawdumpp	sda11 / sde11	volatile crash-dump scratch

1. Obtain the official V311 firmware

The project used firmware **V311**, and proved it byte-for-byte: the device's own xbl/abl images are identical to those extracted from the V311 OTA, whose payload matches the hash declared by the package and whose header is signed by the Nubia OTA certificate (O=nubia). The Europe **V411** package is a different build with different signing certificates (leaf 5c2aaa9b... vs V311's 6ca946b4...) and different xbl/abl hashes — it is the wrong firmware for this (NA/Asia) unit. Use V311.

Verified in the repo

V311 is the correct version for NX679J-UN. The build id in the device dump is the V311 build (eng.nubia.20220308.193302, subversion nx679j_un). Do not substitute V411 unless your device is genuinely the Europe variant.

1.1 Download from the official Nubia host

The OTA is a ~3.9 GB ZIP. The official URL recorded in the project's own wget log:

```
# official Nubia host (URL from the project's V311-download.log)
wget -c "https://rom.download.nubia.com/Europe%26Asia/NX679J/V311/NX679J-update.zip"

# (for reference only – the Europe build, NOT for this unit)
# https://rom.download.nubia.com/Europe/NX679J/V411/NX679J-update.zip
```

Reconstruction note

There is no official public hash of the OTA to compare against — the project flags this as its main provenance gap. If the Nubia host is unreachable, the same V311 ZIP is mirrored on firmware aggregators (halabtech, firmwarepanda, needrom) behind registration; treat any mirror as untrusted until its extracted payload matches what the device itself reports.

1.2 Extract the images

The ZIP contains an A/B OTA payload.bin. Extract it with payload-dumper-go (github.com/ssut/payload-dumper-go):

```
unzip -o NX679J-update.zip payload.bin payload_properties.txt META-INF/com/android/otacert
payload-dumper-go -o extracted payload.bin
# → extracted/{boot,vendor_boot,dtbo,vbmeta,vbmeta_system,recovery,super}.img ...

# optional integrity anchor the project used:
sha256sum payload.bin
unzip -p NX679J-update.zip payload_properties.txt | grep FILE_HASH # base64 of the same sha256
```

You now have the stock boot.img (for the Magisk patch in step 3), vendor_boot.img (vendor modules + DTBs), dtbo.img (overlay to patch in step 5), and super.img (contains vendor, from which the touch firmware is taken).

2. Unlock the bootloader

WARNING

Unlocking **erases all user data** on the phone. Back up first. From this point the EDL safety net does not exist on this device — slot A Android is your only recovery.

2.1 Enable developer options and OEM unlocking

On the phone: Settings → About → Software information → More → tap **Build number** until “Developer options enabled”. Then Settings → Developer options → enable **USB debugging** and **OEM unlocking**.

2.2 Reboot to the bootloader and unlock

```
adb reboot bootloader
fastboot devices                # confirm the device is listed
fastboot flashing unlock        # confirm on-screen with Volume + Power
# the ABL also exposes the Nubia-specific variant, if needed:
# fastboot oem nubia_unlock NUBIA_NX679J
fastboot reboot
```

The device’s ABL fastboot implements flashing unlock / flashing lock and oem nubia_unlock / oem nubia_lock (decoded from the ABL command table in the project). After unlocking, the device reports unlocked:yes, orange boot state.

Reconstruction note

The ABL command table has **no** fastboot boot (boot-from-RAM), **no** oem edl and **no** reboot edl: the project verified these return “unknown command”. So you cannot test an image with fastboot boot — every image must be flashed to a slot. This is also why there is no quick fastboot route into EDL.

2.3 What unlocking does and does not change

- AVB verification is skipped because the device is unlocked (the ABL prints “Device is unlocked, Skipping boot verification”), which is why an unsigned slot-B image boots at all.
- secure:yes stays true — it describes the firmware signing chain (fused OEM key), not the unlock state. The two coexist.
- The fingerprint sensor typically needs recalibration after unlock, and OTA updates stop working — standard consequences on this family.

3. Root slot A and pull the proprietary blobs

Slot A becomes both the recovery anchor and the source of the proprietary pieces the port needs. None of these blobs are in the repository — they are vendor-owned and must come from your own device or the V311 OTA.

3.1 Root slot A with Magisk

Patch the stock `boot.img` (from step 1) in the Magisk app, then flash it to slot A. The project used Magisk 30.7; its patched image has sha256 `0cd94d56...` and is the known-good recovery image.

```
# copy stock boot.img to the phone, patch it in the Magisk app, pull it back as
# magisk_patched.img, then:
adb reboot bootloader
fastboot flash boot_a magisk_patched.img      # slot A = rooted Android
fastboot set_active a
fastboot reboot
adb shell su -c id                          # expect uid=0(root)
```

Verified in the repo

Keep this patched image and a cold dd backup of the pristine `boot_a`. Together with the V311 OTA they are your entire recovery kit, since EDL is unavailable.

3.2 Pull partition images (root shell)

```
BN=/dev/block/bootdevice/by-name
for p in boot_a boot_b dtbo_a dtbo_b vendor_boot_a vbmeta_a; do
  adb shell su -c "dd if=$BN/$p of=/sdcard/$p.img bs=1M" ; adb pull /sdcard/$p.img
done
sha256sum *.img      # boot/vendor_boot must be 100663296 B
```

3.3 Pull vendor kernel modules (.ko)

The port loads closed vendor modules inside the chroot (display SDE, touch, Wi-Fi cnss, modem rmnet/IPA, the PON key chain). The project kept 283 of them in `stock-modules/`. Two sources:

```
# A) from the live device (root):
adb shell su -c 'tar -C /vendor/lib/modules -cf /sdcard/vendor_modules.tar .'
adb pull /sdcard/vendor_modules.tar
# B) from the OTA vendor_boot ramdisk (header v4):
unpack_bootimg --boot_img vendor_boot.img --out vb_out
# vb_out/vendor_ramdisk00 contains lib/modules/<kernel>/
```

3.4 Pull the Goodix touch firmware

Without these the touch controller stays in ROM mode (`"rom_pid:BERLIN"`) and the screen does not respond:

```
# from the live device (root):
adb pull /vendor/firmware/goodix_firmware.bin      # 182528 B
adb pull /vendor/firmware/goodix_cfg_group.bin     # 4614 B
# or from the OTA vendor.img (needs e2fsprogs):
# debugfs -R "dump /firmware/goodix_firmware.bin gfw.bin" vendor.img
```

Reconstruction note

The persistence tars (`etc/luci/tools`) already contain most of the OpenWrt userland — and `tools.tar` carries the entire prebuilt QRTR modem toolkit and the native-UI support files. So for a v10-level boot you mainly need the vendor `.ko` and the Goodix firmware; for modem/UI parity you additionally extract the persist tars. The only thing you must rebuild is the live-rootfs base (section 4.4).

4. Rebuild a complete boot_b image

This is where the repository stops being a turnkey pipeline. The authors' final builder `build-v90.py` is present and readable, but it asserts on several inputs that are **not published**. The honest path is two-level: first reach a booting OpenWrt with SSH from the artifacts that are present (4A), then approach full parity knowing which pieces you must supply yourself (4B).

`boot_b` = fixed 96 MiB (100,663,296 B). Only the ramdisk changes; header `ramdisk_size` is patched in place.



OVL/owrt = OpenWrt 25.12.5 rootfs + persistence tars (etc/luci/tools) + vendor .ko + goodix firmware + native UI

Figure 3 — Anatomy of `boot_b`. The 96 MiB container and vendor kernel are reused unchanged; only the lz4 ramdisk is rebuilt, and the header's `ramdisk_size` field is patched in place.

4.1 Toolchain

```
# Debian/Ubuntu host
sudo apt install gcc-aarch64-linux-gnu lz4 cpio device-tree-compiler \
    android-sdk-libsparse-utils e2fsprogs python3
# the project built static binaries with aarch64-linux-gnu-gcc 16.1.0, -static -Os
```

4.2 Get the OpenWrt 25.12.5 rootfs

The build overlays an OpenWrt `armsr/armv8` rootfs under `/owrt`. The project recorded the exact artifact: `owrt-aarch64-rootfs.tar.gz`, 5,853,270 B, sha256 `2aaddba9...`, 1307 entries, OpenWrt 25.12.5 (musl). Fetch the official generic rootfs:

```
wget https://downloads.openwrt.org/releases/25.12.5/targets/armsr/armv8/\
owrt-25.12.5-armsr-armv8-generic-rootfs.tar.gz
```

Reconstruction note

The project's copy was a snapshot pre-populated with extra packages (ModemManager, patched `libqmi`, LuCI protocol handlers). The clean official rootfs will be missing those; you add them from the persistence tars and from the OpenWrt feeds listed in the repo's `distfeeds.list` (base, luci, packages, telephony for `aarch64_generic`).

4.3 Level A — reach a booting OpenWrt with SSH (reconstructable)

The init chain that actually boots this device is fully present and builds from the OTA plus the Magisk image. Run the authors' builders in order; each verifies the previous stage by hash:

```
# all under 05-bootchain-re/20260916-native-baseline/ and 02-sorgenti/
python3 build-candidate-v6.py # stock kernel + minimal static-init gadget (USB up)
python3 build-candidate-v7.py # (probe-v7-sonda) measured-good ramdisk
python3 build-candidate-v8.py # PID 1 supervisor + worker (8 syscalls only)
python3 build-candidate-v9.py # + read-only relay on tcp 10.0.0.1:9999
python3 build-v10.py          # + OpenWrt rootfs under /owrt, chroot handover, SSH
```

Before running them, fix the hard-coded paths (all builders assume the author's home):

```
# every builder begins with absolute paths like:
# OUT = Path('/home/user/nx679j-stock/experiments/...')
# MAGISK = Path('/home/user/nx679j-stock/magisk_patched-30700_Zx2eF.img')
# repoint them to your tree, e.g.:
```

```
export NXR00T=$HOME/nx679j
grep -rl "/home/user/nx679j-stock" . | xargs sed -i "s#/home/user/nx679j-stock#$NXR00T#g"
```

Also satisfy the hash guards: build-candidate-v6/7 check the Magisk image (MAGISK_SHA=0cd94d56...) and the stock kernel (f0aa949c...). If your V311 pull differs, update these constants to *your* measured values — never bypass the check, it is what stops a wrong image reaching the device. build-v10.py generates an SSH keypair (~/.ssh/nx679j_key) and bakes the public key into the image. Output: boot_b-init-v10.img, a full 100,663,296 B image that boots OpenWrt and answers SSH on 10.0.0.1.

Verified in the repo

Level A is faithful: build-v10.py and its dependencies (v9.cpio/new-uid0.cpio are produced by the v6–v9 chain, not needed as prebuilt files) are all in the repo. This is the milestone you can actually reproduce.

4.4 Level B — toward the full image (what build-v90.py needs)

build-v90.py starts from a live rootfs snapshot and injects the modem/UI/Wi-Fi payload *after* the persistence tars. These inputs are what it asserts on:

Input build-v90.py requires	In repo?	How to obtain / note
owrt-live19.tar.gz (live rootfs base)	No	The one real blocker. Build it: official 25.12.5 rootfs + packages from the repo's distfeeds.list, then extract the persist tars over it. Or snapshot a running Level-A device: <code>tar czf owrt-live.tar.gz -C / owrt</code>
boot_b-live.img (container base)	No	Use your boot_b-init-v10.img (from Level A), or any 96 MiB v4 boot_b readback
v9.cpio, new-uid0.cpio	Produced by Level A	Generated by the v6-v10 chain — not needed as separate files
switch-v58-live.sh	Equivalent present	Use switch-v60-persist.sh or build-v64/overlay/nx679j/switch.sh (both carry the PATH fix, the v57 Wi-Fi block and the wrapper the asserts check)
persist tars etc/luci/tools	Yes	04-persistenza/*.tar (342/181/46 entries), sanitised. tools.tar carries the whole QRTR modem toolkit prebuilt
native UI binaries (nx679j-ui, kiosk3, uinput-touch)	Yes — prebuilt	Shipped in 02-sorgenti/wifi-luci/ (497 / 882 / 426 kB) with full source; no recompile needed
qmicli-full, libqmi-glib.so.5.11.0	No (patches yes)	Only needed for carrier-aggregation display, not for data. 14 patches are in qti-patches/; rebuild libqmi from source + patches, or just use the stock qmi-utils/uqmi package

GAP — not in the repository

The one input with no recipe is the live rootfs base `owrt-live19.tar.gz` — a frozen snapshot of the authors' running system. A byte-identical v90 is therefore not possible from the repo alone. But a *functionally equivalent* one is: the final modem design is driven by the QRTR toolkit in `tools.tar` (all prebuilt), the UI binaries are shipped, and the libqmi patches are in `qti-patches/`. So the substitute is narrow: take your Level-A rootfs, extract the three persistence tars over it (which supply the modem toolkit, LuCI pages and UI), add the handful of feed packages the config references, feed that as the live base, and update the size/hash asserts in `build-v90.py` to match. This is reconstruction, not guesswork.

4.5 Flash-time integrity (keep every gate)

The project's flash scripts (`nx679j-flash-v128.sh`) are not in the repo, but the protocol is fully documented and is reproduced in section 5. The essential rule from a real incident: a single flash "looked OK" but was only the block-device cache — the old image was still there after reboot. Always do the double cold read.

5. Flash the image to slot B

WARNING

Flash only to **boot_b** / **dtbo_b**. Never flash slot A while it is your only recovery. Verify every write with a read-back hash before you reboot.

5.1 Patch and flash dtbo_b (panel fix, one-time)

On slot B the default-panel node in DTBO fragment #35 points at the wrong panel (R66451), so the display stays dark. The fix rewrites one entry to the real panel and re-flashes dtbo_b. Decode the overlay, edit entry #35, rebuild, flash:

```
# decode & extract entry 35 (project script: re/dtbo-parse.py)
python3 dtbo-parse.py dtbo.img 35 # writes work/dtbo_entry35.dtb
dtc -I dtb -O dts work/dtbo_entry35.dtb -o e35.dts
# edit e35.dts: qcom,dsi-default-panel
# <&dsi_r66451_amoled_cmd> -> <&dsi_nubia_r6130_amoled_cmd_dphy>
dtc -I dts -O dtb e35.dts -o e35-mod.dtb
# splice it back, preserving header/other entries/padding (helper below)
python3 dtbo-replace.py dtbo.img 35 e35-mod.dtb dtbo_b-mod.img
fastboot set_active b
fastboot flash dtbo_b dtbo_b-mod.img
```

Reconstruction note

The node edit above (the dtc step) is where the panel is actually changed. dtbo-replace.py only re-inserts that already-edited fragment into entry 35: it is a reconstruction (the authors' own splice script is not published), listed in full in Appendix A, and it keeps the header, the other entries and the partition padding intact. Before editing, confirm with dtbo-parse.py that entry 35 really carries the default-panel node on your unit; after writing, verify the round-trip with dtbo-parse.py.

5.2 Flash boot_b — the strong protocol

The authors stopped using fastboot flash for the ramdisk and wrote boot_b from a device shell, because only that let them verify the write with a cold read. Two equivalent routes:

Route A — fastboot (simplest, first-time):

```
fastboot set_active b
fastboot flash boot_b boot_b-init-v10.img
fastboot reboot
```

Route B — in-place from a device shell (the project's "strong" protocol):

```
# push the image, then on the device (root shell over adb/ssh):
N=/dev/sde41 # boot_b, node 259:25
[ -b $N ] || { rm -f $N; mknod $N b 259 25; } # recreate the node every boot
dd if=/tmp/boot_b.img of=$N bs=4M oflag=direct conv=fsync
sync -f $N; sync
dd if=$N bs=1M count=96 2>/dev/null | md5sum # COLD READ 1
sleep 3
dd if=$N bs=1M count=96 2>/dev/null | md5sum # COLD READ 2
# both MUST equal the host md5 of the image. Only then reboot.
```

WARNING

If the two cold reads disagree, or differ from the host hash, **do not reboot**. A reboot onto a half-written boot_b with EDL dead is exactly the brick scenario. Re-write and re-verify first.

The gate chain the project enforced, in order: (1) cold-read backup of the current boot_b with its md5; (2) compare against the expected value and stop if it differs; (3) compare the host image md5; (4) re-verify the md5 after copying to the device; (5) write with oflag=direct conv=fsync

+ sync -f; (6) two cold reads must both match; (7) reboot is a separate, deliberate step the script never does for you.

6. Boot OpenWrt on slot B

6.1 Switch to slot B and boot

```
adb reboot bootloader          # or, from OpenWrt later: reboot2 bootloader
fastboot set_active b
fastboot continue              # or: fastboot reboot
```

6.2 What happens during boot

The ABL hands the vendor kernel its custom ramdisk. PID 1 (a tiny static init, 8 syscalls only, that can never exit) brings up the USB gadget and then execs the `ramdisk switch.sh`, which prepares and enters the chroot:

- mounts `proc/sys/tmpfs` and makes the device nodes (the stock kernel has no `devtmpfs`);
- loads the `vfat` + `nls` modules, mounts the modem firmware, nudges the `remoteproc`s;
- runs **PERSIST-INSIDE**: extracts `etc/luci/tools` tars over `/owrt`;
- starts `ubusd` and `dropbear`, brings up Wi-Fi, then `chroot /owrt /sbin/init (procd)`, staying alive forever so a dead PID 1 never panics the kernel.

6.3 Connect and verify

```
# USB gadget (preferred, ~3 ms): host gets a new NCM interface
ssh -i ~/.ssh/nx679j_key -o IdentitiesOnly=yes -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null root@10.0.0.1
# Wi-Fi AP "NX679J-TEST" (5 GHz): device at 192.168.77.1
# LuCI: http://10.0.0.1/ (HTTP 200 once procd is up)
```

Reconstruction note

The `dropbear` host key is regenerated every boot, hence `UserKnownHostsFile=/dev/null`. The modem comes up on its own at roughly 6 minutes after boot in the authors' final image; a plain Level-A image gives you SSH and LuCI but not the full modem/UI stack.

6.4 Reboots from inside OpenWrt

```
reboot -f                      # busybox reboot does NOT cross the chroot; use -f
# or: echo b > /proc/sysrq-trigger
reboot2 bootloader             # project helper: RESTART2 → ABL → fastboot, no key press
```

WARNING

Never commit to a DRM display on an unknown CRTC state, and never power down the DSI link — both hang the panel into an unkillable state whose only exit is a hardware power-cycle (long-press Power ~15–20 s). These are documented hard rules; if you run the native UI, read the project's display notes first.

7. Return to Android on slot A

Slot A is untouched throughout, so returning to stock Android is just an active-slot switch — no flashing.

7.1 From the bootloader or Android

```
adb reboot bootloader          # from Android or OpenWrt (reboot2 bootloader)
fastboot set_active a
fastboot continue              # Android boots from slot A (rooted, Magisk)
```

7.2 From inside OpenWrt, hands-free

```
reboot2 bootloader            # → ABL fastboot
# then from the host:
fastboot --set-active=a && fastboot continue
```

Verified in the repo

The project switched A↔B hundreds of times this way. Slot A is the “oracle”: boot it to watch how the stock driver/HWC drive the hardware, then switch back to B. Verify a reboot actually happened by uptime ($U2 < U1$), not by the command returning.

7.3 Confirm you are on Android

```
adb shell getprop ro.boot.slot_suffix    # _a
adb shell su -c id                      # uid=0(root) (Magisk)
```

Keep the phone on slot A whenever you are not actively testing slot B. It is the state with a working recovery, and it is where the fingerprint, calibrations and modem EFS are known-good.

8. Revert and safe recovery

Two distinct goals. 8A undoes the OpenWrt experiment but *keeps* the phone rooted with an unlocked bootloader. 8B returns the device to full stock: no root, bootloader relocked.

8.1 Undo slot B only (keep root + unlocked bootloader)

Because slot A is independent and untouched, “undoing” the port is simply: stop booting B, and optionally restore B to a stock boot image so nothing custom remains.

```
# a) just stop using B – stay on Android:
fastboot set_active a && fastboot continue

# b) optionally wipe the port from B by restoring the stock boot_b + dtbo_b
#    pulled from the device (section 3.2):
fastboot set_active b
fastboot flash boot_b extracted/boot.img
fastboot flash dtbo_b extracted/dtbo.img      # undoes the panel-node edit
fastboot set_active a
fastboot reboot
```

Verified in the repo

This leaves exactly a normal rooted RedMagic 7: unlocked bootloader, Magisk on slot A, both slots carrying stock boot images. Nothing from the port survives.

8.2 Full stock restore (remove root, relock bootloader)

WARNING

Relocking a device whose slots are not 100% stock-signed can make it refuse to boot — and with EDL dead there is no rescue. Relock **only** after every partition you touched is back to the original on device, verified by hash, and the device boots stock cleanly on both slots.

Step 1 — restore every modified partition to stock V311:

```
fastboot set_active a
fastboot flash boot_a extracted/boot.img      # remove Magisk from A
fastboot flash boot_b extracted/boot.img
fastboot flash dtbo_a extracted/dtbo.img
fastboot flash dtbo_b extracted/dtbo.img
# if you ever wrote vendor_boot/vbmeta, restore those from the OTA too
fastboot reboot                               # confirm clean stock boot, both slots
```

Step 2 — clear any stray BCB and confirm state:

```
# from stock Android (still rooted at this point), clear misc to avoid an EDL loop:
adb shell su -c 'dd if=/dev/zero of=/dev/block/by-name/misc bs=32 count=1 conv=notrunc'
adb reboot bootloader
fastboot getvar unlocked                      # expect: yes (still unlocked at this stage)
```

Step 3 — relock (last, irreversible-feeling, data-wiping):

```
fastboot flashing lock                       # or: fastboot oem nubia_lock NUBIA_NX679J
# confirm on-screen; the device wipes data and relocks
fastboot reboot
```

Reconstruction note

The project itself never relocked — it deliberately kept slot A rooted as the recovery anchor and treated relock as out of scope. Step 8.2 is reconstructed from the ABL command table (flashing lock, oem nubia_lock are present) and standard practice, not from a procedure the authors executed. Treat it as the highest-risk part of this guide.

8.3 If a boot goes wrong

Symptom	Action
Phone stuck on logo after flashing B	Volume Down + Power from off → fastboot; set_active a; continue
Device shell alive but frozen UI / 100% ping loss	Hardware power-cycle: long-press Power ~15–20 s (PMIC reset)
Fastboot bootloop / EDL loop (05c6:9008)	Boot slot A Android, clear misc BCB (8.2 step 2); do NOT send blind EDL commands
Nothing enumerates at all	This is the brick case EDL would normally cover — which does not work here

Appendix A — dtbo-replace.py (reconstruction)

What this script does — and does not do. It performs only the *container* operation: it drops a ready-made .dtb into entry N of the DTBO image and leaves everything else untouched. It works at the byte level on the dt_table and never looks inside the fragment: it does **not** know what a panel is, does **not** search for dsi_r66451_amoled_cmd, and does **not** rename any node. The actual panel edit — changing qcom,dsi-default-panel from <&dsi_r66451_amoled_cmd> to <&dsi_nubia_r6130_amoled_cmd_dphy> — happens *before* this script, in the dtc + hand-edit steps of section 5.1. This script is only the last link: it re-inserts the fragment you already modified.

Pipeline (who does what).

1. dtbo-parse.py dtbo.img 35 — extract the fragment (read side, in the repo).
2. dtc -I dtb -O dts ... — decompile to text.
3. **hand-edit the .dts** — this is where the panel node is actually changed.
4. dtc -I dts -O dtb ... — recompile to e35-mod.dtb.
5. dtbo-replace.py — re-insert that blob (this script).

WARNING

The index 35 and the node names come from the project's write-up, not from a check against *your* DTBO. The device's dtbo.img is a vendor blob that is not in the repository, so before editing, run dtbo-parse.py to find which fragment actually carries qcom,dsi-default-panel and confirm the real &dsi_... names on your unit. What is verified below is only the re-insertion mechanics, not the panel edit itself.

The script (append-only: the new fragment is always written into the partition padding, 4-byte aligned; only the chosen entry's size/offset and the header total_size change; other entries, shared-offset dedup entries, and any custom bytes in the entry slot are left intact):

```
#!/usr/bin/env python3
"""dtbo-replace.py src.img <index> new-entry.dtb out.img
Re-inserts ONE already-prepared .dtb as entry <index> of a DTBO image.
Does NOT edit node contents — prepare new-entry.dtb with dtc first."""
import struct, sys, hashlib
if len(sys.argv) != 5: sys.exit(__doc__)
src, idx, newp, out = sys.argv[1], int(sys.argv[2], 0), sys.argv[3], sys.argv[4]
d = bytearray(open(src, 'rb').read()); new = open(newp, 'rb').read()
magic, total, hsz, esz, cnt, eoff, psz, ver = struct.unpack_from('>8I', d, 0)
assert magic == 0xd7b7able, 'not a DTBO: magic 0x%08x' % magic
assert new[:4] == b'\xd0\x0d\xfe\xed', 'new entry is not an FDT'
assert esz >= 16 and 0 <= idx < cnt and eoff + cnt*esz <= len(d)
assert total <= len(d), 'total_size beyond file'
e = eoff + idx*esz
old_size, old_off = struct.unpack_from('>2I', d, e)
shared = [i for i in range(cnt) if i != idx and
          struct.unpack_from('>2I', d, eoff + i*esz)[1] == old_off]
no = (total + 3) & ~3 # append, 4-byte aligned
end = no + len(new)
assert end <= len(d), 'no room in partition padding'
d[no:end] = new
struct.pack_into('>I', d, 4, end) # header total_size
struct.pack_into('>2I', d, e, len(new), no) # this entry: size, offset
open(out, 'wb').write(d)
print('entry %d: %d->%d B, moved @0x%x (was @0x%x)%s'
      % (idx, old_size, len(new), no, old_off,
        '; dedup kept for %s' % shared if shared else ''))
print('file %d B (unchanged); total_size %d; sha256 %s'
      % (len(d), end, hashlib.sha256(d).hexdigest()))
```

Reconstruction note

This helper is a reconstruction, written and tested for this guide: the repository ships the read-side tool (`dtbo-parse.py`) but not the authors' write-side splice script. Its re-insertion mechanics were checked on a 44-entry / 24 MiB DTBO with entry size 16 and 32, for a smaller and a larger replacement: output size preserved, only the target entry changed, every other fragment byte-identical, shared-offset (dedup) and custom entry bytes intact, and the result re-parsed cleanly by `dtbo-parse.py`. An earlier in-place version could overrun neighbouring fragments when the declared entry size was wrong; this append-only version cannot. Always re-verify the round trip with `dtbo-parse.py`, then the two cold-read hashes of section 5.2, before you reboot.

Appendix B — Honest gap list

What	Status	Consequence
owrt-live19.tar.gz (live rootfs base)	Not in repo, no recipe	The one real blocker to a byte-identical v90; rebuild a functional equivalent (section 4.4)
libqmi-glib.so.5.11.0 (prebuilt)	Binary absent; 14 patches present	Rebuild from source + qti-patches/, or use stock qmi-utils/uqmi; needed only for the MM route, which the final design drops
qmicli-full	Not in repo	Carrier-aggregation display only; data works without it
Flash scripts (nx679j-flash-v128.sh)	Described, not shipped	Use the manual strong protocol in section 5
DTBO write-side splice script	Not in repo	Use Appendix A reconstruction
Native UI / kiosk3 / uinput-touch	Prebuilt AND source present	No action — shipped in 02-sorgenti/wifi-luci/
QRTM modem toolkit (final datapath)	Prebuilt in tools.tar	No action — the final modem bring-up runs from these
v91-v180 builders	Not in repo	build-v10 and v54-v90 exist; later tweaks are described in the ESITO/STATO docs, not scripted
Official OTA hash	Does not exist publicly	Anchor trust on what the device itself reports, not on a mirror
Relock procedure	Never executed by the authors	Section 8.2 is reconstructed; highest brick risk

Appendix C — Key source files in the repository

Path	What it gives you
ESTRAZIONE-BLOB.md	The authors' blob-extraction procedures (sections 1-10)
01-documenti/V10-BUILD-PLAN.md	The v10 build plan: rootfs hash, init chain, SSH key step
05-bootchain-re/20260916-native-baseline/build-candidate-v6.py	Minimal static-init gadget (USB bring-up)
05-bootchain-re/.../probe-v7-sonda/build-probe-v7.py	Measured-good v7 ramdisk
02-sorgenti/20260917-init-v8/build-candidate-v8.py	PID-1 supervisor + worker
02-sorgenti/20260917-init-v9/build-candidate-v9.py	+ read-only relay
09-albero-originale/experiments-extra/build-v10.py	Full v10: OpenWrt rootfs + chroot + SSH
02-sorgenti/wifi-luci/build-v90.py	Final builder (reference; depends on unpublished inputs)
04-persistenza/{etc,luci,tools}.tar	Sanitised userland (342 / 181 / 46 entries)
02-sorgenti/20260917-boot-chain-re/dtbo-parse.py	DTBO decoder (read side)
08-skill/nx679j-openwrt/references/boot-flash-device.md	Flash protocol, access, partition nodes
05-bootchain-re/.../PROVENANCE-bootloader.md	The V311-vs-V411 byte-for-byte proof
DOCUMENTAZIONE.md §9 "Come ricostruire da zero"	The authors' ordered rebuild path — the backbone this guide follows

Path	What it gives you
02-sorgenti/wifi-luci/payload-build-v64/	Partial injected payload (8 files); not the full rootfs §9 step 3 implies
02-sorgenti/wifi-luci/qti-patches/	The 14 libqmi/ModemManager patches
04-persistenza/tools.tar	Prebuilt QRTR modem toolkit + UI support — the final datapath